Interview Workflow UI Builder Guide

Interview Quick Reference — Table of Contents

Interview Workflow UI Builder Guide
1. Interview Quick Start
2. Master Node Inventory — every live, registered node type
3. Main Demo — 5-Minute Overview
4. Main Demo — Full Drag/Drop Build
5. Node-by-Node Reference — Core Building Blocks & Control & Flow
6. Control & Flow — connective tissue
7. Decision / Router / Multi-Route / Join — the worked examples
8. MCP Tool vs. MCP Agent
9. RAG / Knowledge Retrieval
10. Human Review — in practice
11. Integrations, in practice
12. Subprocesses, in practice
13. Document & Output Nodes
14. Remaining Node Mini-Demos — the proposal-drafting pipelines
15. Testing Cheat Sheet — the main demo (§4)
16. Complete Node Coverage Matrix
17. Interview Talking Points
18. The Companion Workflow YAML
19. Validation Checklist

Interview Workflow UI Builder Guide

How to read this. Everything here is verified against the live code (`app/nodes/*.py` , `app/runtime/*.py` , `ui/src/modes/studio/*.tsx`) as of this writing — not against older docs, which sometimes disagree with the current UI (noted inline where that happens). Palette search terms are the **exact text you type** in the Builder's node palette.

One important correction up front. This platform is not a generic customer-service tool — its "specialized" node families (Proposal Engineering, Research & Discovery, Evidence & Retrieval, Document Rendering & Export) are purpose-built for **EU Horizon-proposal drafting**. But it also ships a small, genuinely generic **Core Building Blocks** set (Start, Transform, Decision, Router, Human Review, Email, Integration, MCP Tool, External Action, End) that the platform's own maintainers use for business-process automation like customer-case routing — see `workflows/pump_manufacturer_case_routing.yaml` , a real production workflow built almost entirely from these 8 nodes. **This guide's main interview demo uses that same pattern** (a customer-message routing workflow), because it's the natural fit for "build a workflow live" and it's exactly what the platform's own docs recommend for that shape of problem. The specialized proposal-drafting families get their own mini-demos later, on their own terms.

1. Interview Quick Start

Page 1 — Workflow mental model

```
CUSTOMER / BUSINESS INPUT        Start
        ↓
UNDERSTAND                       Transform (AI mode)
        ↓
LOOK UP BUSINESS CONTEXT         MCP Tool  /  Knowledge Retrieval  /  RAG
        ↓
APPLY BUSINESS RULES             Decision
        ↓
ROUTE                            Router  (single branch, or Multi-Route)
        ↓
PERFORM WORK                     Transform (deterministic)  /  Email  /
Integration  /  External Action
        ↓
HUMAN CONTROL                    Human Review
        ↓
JOIN (only if branches ran in parallel)   Join  (= TextAssemblerAgent)
        ↓
OUTCOME                          End
```

This is the platform's own stated design principle: **AI understands, rules decide, and every automation boundary (human/external/AI/deterministic) is visible on the canvas** — never hidden inside a prompt.

Page 2 — Which node should I use?

Need	Search this in the palette	Live type	Notes
Start a workflow with a form	`Start`	`StartAgent`	One node, `mode: input_form`
Start a workflow as a chatbot	`Start`	`StartAgent`	Same node, `mode: chatbot`
Understand free text (extract/classify/ summarize/translate/draft/ analyze)	`Transform`	`TransformAgent`, `mode: ai`	**Not** "AI Task" — see the callout below
Reshape/rename/combine data, no AI	`Transform`	`TransformAgent`, `mode: deterministic`	**Not** "Data Transform" — see below
Deterministic yes/no or multi-fact business rule	`Decision`	`DecisionAgent`	IF/THEN, zero LLM calls
Choose exactly one path	`Router`	`RouterAgent`, `selection: single`	
Choose several paths at once	`Router`	`RouterAgent`, `selection: multi`	Same node, a config toggle — not a different node
Combine parallel branches back together	`Join`	`TextAssemblerAgent`	Palette literally shows "Join"

Need	Search this in the palette	Live type	Notes
Human approval / edit / reject	`Human Review`	`HumanInLoopAgent`	
Search a knowledge base, no generation	*(type the class name)* `KnowledgeRetrieval`	`KnowledgeRetrieval`	Retrieval only
Search + generate a cited answer	*(type the class name)* `RAGAgent`	`RAGAgent`	Retrieval + generation
One known business-system operation (CRM/ERP)	`MCP Tool`	`MCPToolAgent`	Author picks the exact tool
Open-ended multi-tool investigation	*(type the class name)* `MCPAgent`	`MCPAgent`	Model picks the tools — higher risk, no policy gate
Reusable sub-workflow	*(type the class name)* `SubprocessAgent`	`SubprocessAgent`	Runs as an independent child run
Send/read/draft email	`Email`	`EmailAgent`	Operation selector, one node for all 5 verbs
Cloud file access (Drive/OneDrive)	`Integration`	`IntegrationAgent`	
Call an arbitrary REST API / webhook	*(type the class name)* `ExternalActionAgent`	`ExternalActionAgent`	No MCP connection needed
Produce a PDF / DOCX / PPTX	*(type the class name, e.g.)* `DOCXProposalRenderer`	see §14	7 renderer/extractor nodes
End the workflow and shape the result	`End`	`EndAgent`	

> **The single most important "gotcha" in this platform's current UI:** the two nodes the older internal docs call "AI Task" (`AITaskAgent`) and "Data Transform" (`DataTransformAgent`) are **hidden from the palette** (`ui/src/modes/studio/NodePalette.tsx` , `HIDDEN_FROM_PALETTE`). They still exist and still run for already-saved workflows, but **you cannot newly drag them in**. Their replacement is one node, **`TransformAgent` , shown in the palette simply as "Transform"**, with a `mode: ai` (replaces AI Task) or `mode: deterministic` (replaces Data Transform) config toggle. Likewise `WorkflowInputAgent` ("Input") is hidden in favor of `StartAgent` ("Start"). If your own mental model says "drag an AI Task node," what you actually do in this UI is **drag Transform and set its mode to `ai`** .

Page 3 — Connection vs. Mapping

These are two different things, done in two different places in the UI.

```
CONNECTION  (draw an edge on the canvas)
  Start → Transform
  This only sets ORDER: Transform is allowed to run after Start.
  It does NOT move any data by itself.

MAPPING  (the node's "Inputs" tab, or a field's picker)
  Transform.input ← {{outputs.start.data.message}}
  This is what actually moves a value from one step into another step's field.
```

A connection with no mapping is a workflow where step 2 runs after step 1 but never reads anything step 1 produced — a common mistake. In the Builder: - **Connection**: drag from one node's dot to the next node's dot. This is standard React Flow — every node has exactly one input dot and one output dot; branch names (Router/Decision) are a **label on the edge**, not a separate physical port. - **Mapping**: open the node's **Inputs tab** (not the Configure tab) — every text-capable config field shows as a card you can leave as a literal value or map to an upstream field via the picker. The picker writes a `{{node_id.field}}` reference for you; you never hand-type the braces unless you're editing raw YAML.

Page 4 — Interview demo palette search sequence

The exact order you'll drag nodes in for the main demo (§2):

```
1  Start
2  Transform          (mode: ai   — "Understand Message")
3  MCP Tool            ("Look Up Customer")
4  Decision            ("Business Rules")
5  Router              (mode: field — "Route to Department")
6  Transform × 3        (mode: deterministic — one per department)
7  Human Review         (the escalation branch)
8  Email                (after a human approves the escalation)
9  End × 4              (one per terminal branch)
```

2. Master Node Inventory — every live, registered node type

Coverage check, per the repository's actual registry (`app/nodes/registry.py` + `app/nodes/categories.py`, cross-verified directly against source — 57 registered classes, 57 category entries, 1:1 match, zero drift):

```
registered UI-authorable demo nodes:            57
documented in this guide (table below + cards):  57
──────────────────────────────────────────────────
difference:                                       0
```

Three of the 57 are **registered but hidden from new authoring** (`AITaskAgent`, `DataTransformAgent`, `WorkflowInputAgent` — superseded by `TransformAgent` / `StartAgent`, kept only so already-saved workflows still open). They're included below and marked **HIDDEN** — excluded from "what to drag" instructions everywhere else in this guide, per the platform's own intent.

Legend — **Kind**: `ai` (a model decides) / `det` (deterministic code) / `ext` (external system changes) / `human` / `in` (data enters) / `out` (data leaves). **AI**: does this node call a model. **Ext**: does this node act outside the platform (send/write/disclose).

2.1 Core Building Blocks — the small, general-purpose vocabulary

Palette name	Live type	Kind	AI	Ext	One-line purpose
Start	`StartAgent`	in	no	no	Canonical entry point — a business form or a chatbot, never both
End	`EndAgent`	out	no	no	Canonical exit point — shapes the workflow's final result
~~Input~~ **(HIDDEN)**	`WorkflowInputAgent`	in	no	no	Predecessor of Start — still works, can't be newly dragged
~~AI Task~~ **(HIDDEN)**	`AITaskAgent`	ai	yes	no	Predecessor of Transform's AI mode
Decision	`DecisionAgent`	det	no	no	Ordered IF/THEN business rules → named conclusions

Palette name	Live type	Kind	AI	Ext	One-line purpose
Router	`RouterAgent`	det (ai if `mode: llm`)	conditional	no	Chooses one branch (or several, in Multi-Route)
~~Transform~~ **(HIDDEN, as DataTransformAgent)**	`DataTransformAgent`	det	no	no	Predecessor of Transform's deterministic mode
Human Review	`HumanInLoopAgent`	human	no	no	Pause and wait for approve / edit / reject
Email	`EmailAgent`	ext	no	yes	One mailbox capability: search/read/ create_draft/ reply/send
Integration	`IntegrationAgent`	ext	no	yes	Cloud storage: list/ search/select/ get a Drive or OneDrive file
MCP Tool	`MCPToolAgent`	ext	no	yes	Call one named tool on one connected business system
(type) `ExternalActionAgent`	`ExternalActionAgent`	ext	no	yes	Generic REST call / webhook when there's no MCP connection

2.2 Control & Flow — connective tissue

Palette name	Live type	Kind	AI	Ext	One-line purpose
(type) `Literal`	`Literal`	in	no	no	Emits a fixed config value — scaffolding/ smoke tests
(type) `Echo`	`Echo`	det	no	no	Renders a template string — isolates templating for testing

Palette name	Live type	Kind	AI	Ext	One-line purpose
Transform	`TransformAgent`	ai (det if `mode: deterministic`)	conditional	no	**The** general-purpose AI/deterministic step — see §1 callout
(type) `WorkflowFileLoader`	`WorkflowFileLoader`	in	no	no	Extracts text from uploaded PDF/DOCX/PPTX/spreadsheets
Join	`TextAssemblerAgent`	det	no	no	Deterministically combines parts from parallel branches
(type) `SubprocessAgent`	`SubprocessAgent`	det	no	no	Runs another saved workflow as an independent child run

2.3 Research & Discovery — finding candidate sources (proposal-drafting)

Palette name (all shown as raw class name)	Kind	AI	Ext	One-line purpose
`BoundedDeepResearchAgent`	ai	yes	yes	Budgeted, multi-brief web-research tool-loop, produces candidates
`WebSearchAgent`	ext	no	yes	One ad-hoc live web query (Auto/Tavily/OpenAI/Kimi)
`ScholarlyCandidateDiscoveryAgent`	ext	yes	yes	Multi-query academic-database fan-out (arXiv, OpenAlex, …)
`ResearchSourceAcquirer`	ext	no	yes	Fetches candidates into immutable, hash-stamped full text
`ScientificResearchPlannerAgent`	ai	yes	no	Turns a call/concept into several bounded research briefs

2.4 Evidence & Retrieval — verifying and serving evidence

Palette name	Kind	AI	Ext	One-line purpose
`RAGAgent`	ai	yes	no	Hybrid retrieval + `[N]`-cited grounded answer
`KnowledgeRetrieval`	det	no	no	Retrieval only, no generation (complements RAGAgent)
`InternalProjectEvidenceRetrieverAgent`	ai	yes	no	Partner/internal facts — requires human approval to use
`PriorProjectRetrieverAgent`	det	no	no	Official-domain search (CORDIS/ LIFE/EIP-AGRI) for prior art
`StructuredDatasetRetrieverAgent`	det	no	no	Bounded, auditable Eurostat/ structured-data retrieval
`PaperQAEvidenceSynthesizerAgent`	det*	yes*	no	PaperQA2 synthesis over already-acquired full text
`MinIOEvidenceIngestion`	ext	no	no	Indexes acquired sources into the vector store for RAG
`ClaimEvidenceVerifier`	ai	yes	no	Verifies claims against exact source passages
`ProposalTruthGraphAgent`	det	no	no	Freezes the drafting-safe, hash-stamped truth graph
`CitationRegistryBuilder`	det	no	no	Deterministic, numbered, renderer-ready citation registry

* `PaperQAEvidenceSynthesizerAgent` calls LLMs internally via PaperQA2's own config, not the platform's shared `llm` service — its manifest badge under-reports this.

2.5 Proposal Engineering — the EU-Horizon-specific pipeline

Palette name	Kind	AI	Ext	One-line purpose
`GraphNormalizer`	det (pinned)	yes	no	The keystone node — concept note → typed proposal graph
`ConceptAlternativesAgent`	ai	yes	no	3 independently-scored concept postures
`ConceptFreezeAgent`	det	no	no	Resolves the human's concept choice — fails closed
`MethodologyEngineeringAgent`	ai	yes	no	One Method Card per objective
`ProposalEvidenceFactoryAgent`	ai	yes	no	Full claim-verification pipeline, fail-closed gates
`ConsistencyChecker`	det	no	no	9 hard rules over the typed proposal graph
`CallCoverageMatrixAgent`	det	no	no	Per-requirement ADDRESSED/PARTIAL/MISSING matrix
`HorizonEvaluationAgent`	ai	yes	no	Cross-provider Excellence/Impact/Implementation scoring
`ProposalSubmissionGate`	det	no	no	Final READY/BLOCKED release gate — never calls an LLM

2.6 Multimodal

Palette name	Kind	AI	Ext	One-line purpose
`KimiVisionAgent`	ai	yes†	no	Analyzes one uploaded image
`OpenAIImageGenerationAgent`	ai	yes†	no	Generates one image from a prompt
`DynamicFigureAgent`	det†	no	no	Scans drafted content for `[[IMAGE PROMPT: ...]]` markers, generates images inline

Palette name	Kind	AI	Ext	One-line purpose
`FigureEmbedder`	det	no	no	Older, explicit-marker image-embedding (superseded by DynamicFigureAgent)

†Manifest badge under-reports `uses_ai` / `execution_kind` for these three (see §12).

2.7 Document Rendering & Export

Palette name	Kind	One-line purpose
`ExcelTableExtractor`	in	Extracts every sheet of an uploaded `.xlsx` into rows/cells
`PDFTextExtractor`	in	Extracts per-page text from an uploaded `.pdf` (no OCR)
`PDFProposalRenderer`	out	Sections → styled, print-ready PDF (WeasyPrint)
`PowerPointProposalSlides`	out	Sections → `.pptx`, one slide per section
`DOCXProposalRenderer`	out	Sections → `.docx`, same contract as the PDF renderer
`HorizonDOCXProposalRenderer`	out	Horizon Part B DOCX: TOC, citations, page-limit awareness
`HorizonHTMLProposalRenderer`	out	Horizon Part B PDF (+HTML source), same domain features

2.8 Integrations — protocols, not one specific business system

Palette name	Kind	AI	Ext	One-line purpose
(type) `MCPAgent`	ext	yes	yes	Autonomous tool-choosing loop — model picks the tools
(type) `ScientificSkillAgent`	ext	yes	yes	One skill-guided LLM completion for scientific writing
(type) `SQLQueryAgent`	ext	no	no	Read-only `SELECT` against a business-records MCP server
(type) `PythonSnippetAgent`	det	no	no	Runs a short Python snippet in an isolated sidecar

3. Main Demo — 5-Minute Overview

Scenario: Customer Request Routing. A customer sends a free-text message. The workflow reads it, looks the customer up in the connected business system, decides which department owns the case using deterministic rules (never the model), routes there, and produces a clean hand-off packet — escalating to a human first when the extraction is unsure or the customer can't be found, and only then sending an acknowledgement email once a person has approved it.

This mirrors the platform's own production workflow (`workflows/pump_manufacturer_case_routing.yaml`) at demo scale — same shape, five real node families, one preflight-clean file.

```
Start                          customer's message, subject, name
  ↓
Transform  (mode: ai)          "Understand Message" — intent, summary, confidence
  ↓
MCP Tool                       "Look Up Customer" —
dynamics365_finance_scm.find_customer
  ↓
Decision                       "Business Rules" — department + escalation facts
  ↓
Router  (mode: field)          "Route to Department"
  ├─ SALES        → Transform (deterministic) → End
  ├─ ENGINEERING  → Transform (deterministic) → End
  ├─ SUPPORT      → Transform (deterministic) → End
  └─ NEEDS_REVIEW → Human Review → Email (create_draft) → End
```

Node families touched: input (Start), AI understanding (Transform/ai), external read (MCP Tool), deterministic business logic (Decision), branching (Router), deterministic reshaping (Transform/deterministic), human gate (Human Review), external write behind a review gate (Email), output (End). Multi-Route and Join deliberately do **not** appear here — see the callout in §3.4 for why, and §7 for where they belong instead.

3.1 Why this shape, in one sentence per step

Step	What I do	What I say
Start	Drag Start, add a `message` field	"This is the one place a request enters the workflow — everything downstream addresses it the same way regardless of channel."
Transform (ai)	Drag Transform, set mode `ai`, write the extraction instruction	"The model only *understands* the message — it never decides where the case goes."
MCP Tool	Drag MCP Tool, pick `dynamics365_finance_scm` → `find_customer`	"This is a real business-system read, through a policy-gated connection — never a raw API call."
Decision	Drag Decision, write the department + escalation rules	"This is where the actual business logic lives — readable, testable, the same answer every time."

Step	What I do	What I say
Router	Drag Router, mode `field`, map department → branch	“The canvas now shows the four destinations the business actually has.”
Transform (deterministic) × 3	One per department, build the hand-off packet	“No model call here — this is an exact reshape, so there’s no cost, no latency, and no failure mode for something that’s just data shaping.”
Human Review	Gate the uncertain branch	“Anything the rules can’t confidently place stops for a person, instead of guessing.”
Email	`create_draft`, after approval only	“The external action only fires once a human has actually approved something in this run — never asserted, always checked.”
End	One per branch, shape the result	“Every path ends with a clean, typed result — never the whole accumulated state.”

3.2 The four escalation triggers, spelled out

The Decision step sets `department: NEEDS_REVIEW` (overriding whatever the department rules chose) when **either**: 1. `find_customer.found` is `false` — the business system doesn’t recognize this customer, or 2. `understand_message.parsed.confidence` is below `0.6` — the extraction itself is unsure.

Both are real, inspectable facts from real upstream nodes — never a guess bolted on afterward.

3.3 Why Multi-Route and Join are *not* in this workflow

This is a **single, mutually-exclusive** Router: exactly one department branch runs per request. Feeding two branches of one exclusive Router into a shared downstream Join node is a real, named preflight error in this platform (`FANIN_UNREACHABLE_ANDJOIN` / `ROUTER_JOIN_UNREACHABLE`) — the shared node would wait forever for the branch that never ran. Forcing a Join in here wouldn’t just be unnecessary, it would fail validation. Multi-Route and Join get their own honest mini-demo in §7, built on a scenario where they actually apply: a message that genuinely needs **two departments working in parallel**, then combined.

4. Main Demo — Full Drag/Drop Build

14 nodes, in the order you drag them. Each step: what to drag, where to drop it, what to type, what to connect, what to map, what to test.

01 — Start

```
Search:  Start
Place:   first node on the canvas
```

Configure (required):

```
mode: input_form
fields:
  - name: message         type: text     required: true    label: "Customer Message"
  - name: subject         type: string   required: false    label: "Subject"
  - name: customer_name   type: string   required: false    label: "Customer Name"
  - name: customer_email  type: string   required: false    label: "Customer
Email"   format: email
```

Test:

```
message: "My pump keeps overheating after 20 minutes, model PX-400. Can someone call
me? — Anita Rao"
```

02 — Transform (`mode: ai`) — "Understand Message"

```
Search:  Transform
Place:   immediately after Start
```

Configure (required):

```
mode: ai
instructions: |
  Extract what this customer message is asking for. Never invent a fact —
  if the message doesn't say something, return an empty string or UNKNOWN.
output_fields:
  - intent       (enum: NEW_ORDER, TECHNICAL_ISSUE, BILLING_QUESTION,
GENERAL_INQUIRY)
  - summary      (string — one sentence)
  - confidence   (number, 0.0–1.0)
```

Connect: `Start → Transform` **Map:** `Transform.input ← {{outputs.start.data.message}}` **Test:** same message as step 01 → expect `intent: TECHNICAL_ISSUE`, `confidence` ≥ 0.6.

03 — MCP Tool — "Look Up Customer"

```
Search:  MCP Tool
Place:   immediately after Transform
```

Configure (required):

```
server_id: dynamics365_finance_scm
tool:      find_customer
fail_on_error: false
```

Map: `arguments.customer_name ← {{outputs.start.data.customer_name}}` **Connect:** `Transform → MCP Tool` **Test:** a name that exists in the connected system → `found: true`; a made-up name → `found: false, count: 0`.

04 — Decision — "Business Rules"

```
Search:  Decision
Place:   immediately after MCP Tool
```

Configure (required):

```
defaults:
  department: SUPPORT

rules:
  - name: New orders go to Sales
    when:   understand_message.parsed.intent equals NEW_ORDER
    then:   department = SALES

  - name: Technical issues go to Engineering
    when:   understand_message.parsed.intent equals TECHNICAL_ISSUE
    then:   department = ENGINEERING

  - name: Billing questions go to Support
    when:   understand_message.parsed.intent equals BILLING_QUESTION
    then:   department = SUPPORT

  - name: Unrecognized customer needs a human
    when:   find_customer.found is_false
    then:   department = NEEDS_REVIEW

  - name: Low-confidence extraction needs a human
    when:   understand_message.parsed.confidence less_than 0.6
    then:   department = NEEDS_REVIEW
```

Connect: `MCP Tool → Decision` **Test:** the two escalation rules run *last* on purpose — `DecisionAgent` evaluates every rule in order and lets a later rule override an earlier one's conclusion, so "unrecognized customer" or "low confidence" always wins the final `department` value regardless of intent.

05 — Router — "Route to Department"

```
Search:  Router
Place:   immediately after Decision
```

Configure (required):

```
mode: field
route_field: business_rules.decisions.department
branches:
  SALES:        SALES
  ENGINEERING:  ENGINEERING
  SUPPORT:      SUPPORT
  NEEDS_REVIEW: NEEDS_REVIEW
fallback: SUPPORT
```

Connect: `Decision → Router` **Test:** `used_fallback: true` should only ever appear if `department` came back as something none of the four branches name — with the rules above, that shouldn't happen, which is itself worth testing for.

06a/06b/06c — Transform (`mode: deterministic`) — one per department

```
Search:  Transform
Place:   one on each of the SALES / ENGINEERING / SUPPORT branches
```

Configure (required, example shown for the Engineering branch):

```
mode: deterministic
operations:
  - target: department    operation: constant   value: ENGINEERING
  - target: case_summary  operation: copy        source:
understand_message.parsed.summary
  - target: next_action   operation: constant    value: "Identify the unit and
arrange a technician callback."
```

Connect: `Router --ENGINEERING--> Transform` (and the matching branch/config for SALES, SUPPORT) **Map:** each `source:` line above is itself the mapping — no separate step needed once the operation names a source path. **Test:** confirm `data.department` / `data.case_summary` / `data.next_action` all appear in the node's output — an empty `data` here almost always means an operation's `target` was misspelled.

07 — Human Review — the escalation gate

```
Search:  Human Review
Place:   on the Router's NEEDS_REVIEW branch
```

Configure (required):

```
question: "This case could not be routed automatically. Read it and decide."
review_purpose: >-
  Either the customer isn't recognized in the business system, or the
  extraction confidence was too low to act on. Guessing here is worse than
  asking — a case in the wrong queue is usually noticed late.
allowed_actions: [approve, reject]
review_panels:
  - label: "Customer's message"       field: understand_message.parsed.summary
  - label: "Customer found?"          field: find_customer.found
  - label: "Extraction confidence"    field: understand_message.parsed.confidence
```

Connect: `Router --NEEDS_REVIEW--> Human Review` **Test:** there is no explicit "reject" edge to draw — **a reject decision always ends the run immediately**, with no further nodes running. Only `approve` / `edit` continue along whatever edges you draw next. This is a real compiler behavior, not a configuration option.

08 — Email — acknowledge, after approval only

```
Search:  Email
Place:   immediately after Human Review
```

Configure (required):

```
connection: default
operation:  create_draft
to:
  - email: "{{outputs.start.data.customer_email}}"
    name:  "{{outputs.start.data.customer_name}}"
subject: "We're looking into your request"
body:    "{{outputs.understand_message.parsed.summary}}"
```

Connect: `Human Review → Email` **Test:** this only ever runs on `approve` / `edit` — never on `reject` — because of how step 07 behaves. Note the deliberate choice of `create_draft` over `send`: preflight's `EXTERNAL_ACTION_WITHOUT_REVIEW` check specifically watches for a `send` / `reply` with no Human Review gate upstream; `create_draft` sidesteps needing to prove that at all, and is the safer default the platform's own docs recommend.

09a–09d — End × 4

```
Search:  End
Place:   one at the end of each of the four branches (Sales / Engineering /
Support / after Email)
```

Configure (required, example for the Engineering branch):

```
mode: workflow_result
outputs:
  - key: department     value_from: "{{outputs.engineering_case.data.department}}"
  - key: case_summary   value_from: "{{outputs.engineering_case.data.case_summary}}"
  - key: next_action    value_from: "{{outputs.engineering_case.data.next_action}}"
```

Connect: `Transform (deterministic) → End` on each of the three department branches; `Email → End` on the escalation branch. **Test:** run the whole workflow once per branch (see §11's test pack) and confirm each `End` node's `result` contains exactly the three keys above — nothing more, nothing accidentally leaked from earlier steps.

5. Node-by-Node Reference — Core Building Blocks & Control & Flow

Full cards for the 15 nodes you can actually drag in and compose freely (the 3 hidden predecessors get a short note instead of a full card). Every field list below is the real Pydantic config schema — not paraphrased.

Start

Search: `Start` · **Live type:** `StartAgent` · **Family:** core · **Kind:** input

Why: The one entry point every workflow needs — a business form (`input_form`) or a chatbot (`chatbot`), never both on the same node.

```
Required
--------
mode: input_form | chatbot

input_form mode:
  fields: []            (list of field bindings, see below — can be empty but
usually isn't)

chatbot mode:
  (no required fields — all have sensible defaults)
```

```
Optional
--------
input_form:  title, description, file_fields, sample
chatbot:     chatbot_name, welcome_message, message_placeholder,
             allow_attachments (default true), suggested_questions
```

Field binding shape (each entry in `fields`): `name`, `label`, `required`, `type` (string/text/number/integer/boolean/date/object/list), `format` (email/phone/url/currency/percentage/date/time/datetime), `widget` (dropdown/searchable_dropdown/radio/multi_select/checkbox/toggle), `preset` (currency/number_unit/date_range/duration/address/country — pre-fills a whole compound object), `min_length` / `max_length` / `pattern`, and **conditional** `visible_when` / `required_when` — the same AND/OR/NOT condition-group grammar Decision/Router rules use, evaluated against the form's own earlier fields.

Output: `data.<field>` for each declared field, `message` (chatbot mode), `attachments`, `missing` (declared-but-absent required fields — route on this instead of hard-failing if you want to ask the customer for what's missing).

Connects to: always the first node. Downstream steps read `{{outputs.<start_id>.data.<field>}}`.

Interview sentence: *"Every workflow starts here — it's what makes the mapping picker typed from the very first node, instead of beginning mid-air at an AI step."*

Transform

Search: `Transform` · **Live type:** `TransformAgent` · **Family:** specialized (despite being the actively-recommended general-purpose node) · **Kind:** `ai` by default, `deterministic` when `mode: deterministic`

Why: One node, two modes — this is the current, non-deprecated home for both "have the model do something to this text" (replaces the hidden AI Task) and "reshape this data exactly, no model" (replaces the hidden Data Transform).

`mode: ai` (default):

```
Required
--------
instructions: str          (what you want the model to do)

Optional
--------
model: "claude-sonnet-4-5" (default)
input_fields: []            (new-style: named, typed inputs — cleaner than one big
"input" string)
output_fields: []           (FieldSpec list — the structured contract; omit for
free-text output)
language: {...}             (input/output language policy)
temperature: 0.2 (default) · max_tokens: 16384 · max_retries: 1 · fail_on_error:
true
```

Output (ai mode): `raw` (model's raw text), `parsed` (structured, per `output_fields`), `status` (`ok` / `refused` / `invalid_output` / `provider_error`), `error`. > With no `output_fields` at all, `parsed` is statically `{}` — read `.raw` instead in that case, or you'll hit a preflight error (`TEMPLATE_STATICALLY_EMPTY_FIELD`) the moment you reference `.parsed.anything`.

`mode: deterministic`:

```
Required
--------
operations: []    (list of TransformOperation — see the full grammar in §6)

Optional
--------
omit_empty: false   (drop keys whose computed value ended up empty/None)
```

Output (deterministic mode): `data.<target>` for each operation, `defaulted` (targets that fell back to a configured default — surfaced deliberately, since a mapping that quietly nulls out is one of the hardest bugs to spot).

Input: `input ←` (ai mode, legacy) or each `input_fields[].value ←` (ai mode, new-style) or each operation's `source ←` (deterministic mode) — always a `{{...}}` reference to an upstream node.

Connects to: almost anywhere — this is the platform's workhorse. AI mode typically follows Start; deterministic mode typically precedes End.

Quick test: `mode: ai`, instruction "extract the customer's stated urgency as low/normal/high" against *"This is extremely urgent, please call today"* → expect `parsed.urgency: high`.

Interview sentence: *"I use Transform in AI mode to turn free text into structured facts, and the exact same node in deterministic mode to reshape those facts into a clean hand-off — no model call, no cost, no failure mode, for something that's just data shaping."*

Decision

Search: `Decision` · **Live type:** `DecisionAgent` · **Family:** core · **Kind:** deterministic

Why: Ordered IF/THEN business rules (nested AND/OR/NOT), zero LLM calls, same answer every time, every run records exactly which conditions were checked.

```
Required
--------
(none strictly required — but an empty rule set does nothing useful)

Optional
--------
rules:  []     — each: name, when (ConditionGroup), then (list of {field, operation,
value}),
                  default (bool, "otherwise"), stop_on_match (bool), description
defaults: {}   — baseline values present before any rule runs
declared_fields: []  — extra output field names you promise even if no rule sets
them
```

Behavior: all rules evaluate **in order**, not first-match-wins — a later rule's `then` can override an earlier rule's conclusion for the same field, which is exactly how you make an "escalate regardless of department" rule win last (see §4, step 04). Facts set by an earlier rule are visible to later rules under `decisions.<field>`.

Operators: `equals` `not_equals` `contains` `not_contains` `greater_than` `less_than` `greater_or_equal` `less_or_equal` `exists` `does_not_exist` `is_empty` `is_not_empty` `in` `not_in` `is_true` `is_false`.

Output: `decisions.<field>` (the resolved facts — what downstream nodes reference), `matched_rules` (which rule names fired), `explanation` (full per-condition trace), `summary` (human-readable lines).

Connects to: after a Transform (ai mode) understanding step; before a Router.

Interview sentence: *"This is where the actual business policy lives — readable by anyone, testable in isolation, and it gives the same answer on the same facts every single time."*

Router

Search: `Router` · **Live type:** `RouterAgent` · **Family:** core · **Kind:** deterministic (ai only in `mode: llm`)

Why: Chooses which downstream branch(es) run, and records why.

```
Required (mode-dependent)
--------
mode: field | conditions | rule | llm

field mode:       route_field, branches (value → route-name dict)
conditions mode:  cases (ordered list of {route, when: ConditionGroup, description})
rule mode:        rules (legacy string-expression, kept for old workflows)
llm mode:         model, prompt

Optional
--------
fallback: <route name>     — ALWAYS set this on field/conditions mode
selection: single (default) | multi   — "multi" = Multi-Route, see §7
```

Output: `route` (the/first selected branch), `routes` (the authoritative list — always populated, even in single mode), `reason`, `route_value`, `explanation`, `matched_conditions`, `used_fallback` (the single most useful field when a live run routes somewhere unexpected).

Connects to: after Decision; fans out to one node per named branch. The branch name is a **label on the canvas edge**, not a separate handle — the router still has one physical output dot.

Quick test: `used_fallback: true` on a run you expected to hit a named branch almost always means the branch dict's *key* doesn't exactly match the upstream field's *value* (case-sensitive).

Interview sentence: *"A new department, label, or threshold is a configuration change here — never a new node type."*

Human Review

Search: `Human Review` · **Live type:** `HumanInLoopAgent` · **Family:** core · **Kind:** human

Why: Pauses the run and waits for a person — the visible gate in front of any external action or uncertain case.

```
Required
--------
question: str

Optional
--------
review_panels: []       — [{label, field, hint, editable}] — the labelled cards a
reviewer sees
context_fields: []      — raw fallback fields, only used if review_panels is empty
review_purpose: str     — why this gate exists, shown above the decision buttons
editable_content_field  — state path the rich-text editor edits, if any
allow_document_override: true   — let an uploaded document replace the editor
content
max_edit_chars: 1,000,000
allowed_actions: [approve, reject, edit]   (default: all three)
```

Outcomes — exactly three, never more:

```
AI / Rule
   ↓
Human Review
   ├─ approve   → all outgoing edges fire, unchanged content
   ├─ edit      → all outgoing edges fire, with edited_content in place of content
   └─ reject    → the run ends immediately — no outgoing edge fires, ever
```

Output: `decision` (`approve` / `reject` / `edit`), `reason` (on reject), `content` / `edited_content`, `content_overridden` (true if a document upload replaced the editor content).

Connects to: downstream of any uncertain Decision/Router outcome, upstream of any Email `send` / `reply` or MCP Tool write.

Interview sentence: *"This is what makes 'a person decides what may happen automatically' visible on the canvas — as a gate in front of the action, not buried inside a prompt."*

Email

Search: `Email` · **Live type:** `EmailAgent` · **Family:** core · **Kind:** external, always flags `external_action`

Why: One node, one operation selector, instead of a node type per mailbox provider per verb.

```
Required
--------
connection: str                — a configured mailbox connection, never a token
operation:  search | read | create_draft | reply | send

Operation-specific (validated — each op needs its own minimum):
  search:        query, from_address, subject_contains, unread_only,
has_attachments,
                 folder (default "inbox"), newer_than_days, max_results (default 10)
  read / reply:  message_id, thread_id   (reply needs at least one of the two)
  write ops:     to / cc / bcc (recipient lists), subject, body, body_html
                 (create_draft/send need ≥1 recipient AND a body)
```

Output: `operation`, `provider`, `messages` / `message` (flattened summaries — subject/body/from/to/received_at — never raw attachment bytes), `message_count`, `draft_id`, `sent_message_id`, `deduplicated`.

Connects to: `search` / `read` are read-only, safe anywhere. `create_draft` / `reply` / `send` are external actions — preflight's `EXTERNAL_ACTION_WITHOUT_REVIEW` check specifically flags a `send` / `reply` with no Human Review gate guaranteed upstream.

Interview sentence: *"Prefer create_draft over send unless a human review gate genuinely precedes this node — that's not a style preference, preflight actually checks for it."*

Integration

Search: `Integration` · **Live type:** `IntegrationAgent` · **Family:** core · **Kind:** external

Why: One cloud-storage capability (Google Drive / OneDrive), operation-selected, same pattern as Email.

```
Required
--------
provider:   google_drive | onedrive
connection: str
operation:  list_folder | search_files | select_file | select_folder | get_file

Optional
--------
folder_id, file_id, query (default ""), page_size (default 25, 1–200), page_token
```

Output: `status` (ok/error/not_connected/reauth_required), `files` (metadata list), `file`, `first`, `count`, `found`, `downloaded_file` / `downloaded_files` (as `WorkflowFileRef` pointers — the same reference shape as a user upload), `next_page_token`, `error` / `error_code` / `retryable`.

Connects to: typically feeds a `WorkflowFileLoader` next, to turn a downloaded file reference into extractable text.

Interview sentence: *"A downloaded file becomes the same typed reference an uploaded one would be — everything downstream treats them identically."*

MCP Tool

Search: `MCP Tool` · **Live type:** `MCPToolAgent` · **Family:** core · **Kind:** external

Why: The platform's integration primitive — call one named, author-chosen tool on one connected business system (CRM/ERP/anything else exposed via MCP). **Distinct from `MCPAgent`** (§8): here, the author picks the tool; the model never does.

```
Required
--------
server_id: str      (a configured connection — never a URL/credential)
tool:      str

Optional
--------
arguments: {}                       (usually template references)
fail_on_error: true
timeout_seconds (≤600s)
allow_unattended_write: false        (an explicit author statement — does NOT itself
grant write permission)
max_read_retries: 1 (0–3; writes are never retried)
```

Output: `status` (`ok` / `error` / `denied` / `needs_approval` / `skipped`), `data` (the tool's typed result), `first` (first record of a collection, or the single record), `count`, `found` (bool — "did the system know this customer?" with no extra Transform step), `mode` (`live` / `mock`), `error` / `error_code` / `retryable` / `suggested_action`.

> **The found/first pattern:** a required argument that resolves to nothing (e.g. the CRM lookup upstream found no account, so there's no account id) makes the call **`skipped`**, not an error — calling anyway would send a confusing null to the server. Downstream, address `outputs.<node>.first.<field>?` (with the trailing `?`) for anything that may not exist, rather than assuming the call always produced a record.

Connects to: any read/write against a connected business system. A write is refused unless the connection permits it and either `allow_unattended_write` is set or a Human Review has actually approved something earlier in this run (checked against real completed node outputs, never merely asserted).

Interview sentence: *"A new CRM capability is a new tool on the server, discovered automatically — never a new node type."*

External Action

Search: *(type)* `ExternalActionAgent` · **Family:** core · **Kind:** external

Why: A generic REST call or webhook for a system that has no MCP connection at all — kept distinct from MCP's classified, discoverable tools.

```
Required
--------
action_type:  rest_api | webhook
safety_class: read | write | external_action     (no default — deliberate)
url: str

Optional
--------
method: POST (default) · headers: {} · body: Any
timeout_seconds: 30.0 (0–300) · allow_unattended_write: false
```

Output: `status` (`ok` / `error` / `needs_approval`), `safety_class`, `response_status`, `response_body`, `error` / `error_code` / `retryable`.

Connects to: anywhere an MCP Tool doesn't apply — typically gated behind Human Review the same way a write-mode MCP Tool call is.

Interview sentence: *"This is the escape hatch for a system we haven't connected via MCP yet — same safety posture, explicit `safety_class`, no silent default."*

End

Search: `End` · **Live type:** `EndAgent` · **Family:** core · **Kind:** output

Why: The canonical exit point — shapes exactly what a run returns, in one of three modes.

```
Required
--------
mode: workflow_result | chat_response | custom_response

workflow_result:  outputs: []   — [{key, value_from}]
custom_response:  title, message
chat_response:    chat_message, outcome (default "reply"), route_to, route_to_label,
handoff
```

Output: `result` — exactly the shape you declared, nothing more.

Connects to: always a terminal node — declare it in the workflow's `exit:` list.

Interview sentence: *"Every path ends with a clean, typed result — never the whole accumulated run state leaking out."*

Deprecated, hidden predecessors — do not drag these

Hidden node	Superseded by	Why it still matters
`WorkflowInputAgent` ("Input")	Start	Still opens/runs for old workflows; new ones use Start
`AITaskAgent` ("AI Task")	Transform, `mode: ai`	Same idea, Transform has the newer editor + 4 distinct failure statuses
`DataTransformAgent` (labelled "Transform" too)	Transform, `mode: deterministic`	Identical 14-operation grammar, now under one node

6. Control & Flow — connective tissue

Literal

Search: *(type)* `Literal` · **Kind:** input

```
Required: value: Any
Output:   value
```

Interview sentence: *"A fixed stand-in value, useful while the real upstream step isn't wired up yet."*

Echo

Search: *(type)* `Echo` · **Kind:** deterministic

```
Required: template: str
Output:   text
```

Interview sentence: *"Renders a template string against state — good for proving a mapping resolves before you wire the real node behind it."*

Join

Search: `Join` · **Live type:** `TextAssemblerAgent` · **Family:** core (despite living in the Control & Flow category) · **Kind:** deterministic

Why: The platform's business-facing "Join" — deterministically combines already-generated parts from parallel branches, **no LLM call**, specifically so a long final result can never lose content to a token ceiling.

```
Required
--------
parts: []      — one templated reference per branch you're joining, e.g.
                 ["{{outputs.branch_a.text}}", "{{outputs.branch_b.text}}"]

Optional
--------
separator: "\n\n" (default)
```

Output: `text`, `part_count`, `all_parts_present` (false if any listed part resolved empty).

Connects to: the point where two-or-more genuinely parallel branches (a plain fan-out, or a Multi-Route selection) need to reconverge. **Never** the shared target of two mutually-exclusive branches of one single-selection Router — see §3.4 and the worked example in §7.

Interview sentence: *"This waits for every branch it lists, then combines them deterministically — never a model re-assembling an already-drafted document, which risks silent truncation."*

Workflow File Loader

Search: *(type)* `WorkflowFileLoader` · **Kind:** input

```
Required: files: str | WorkflowFileRef | list[WorkflowFileRef]
          (a template placeholder like "{{inputs.source_files}}" is valid at compile
time)
Optional: max_chars_per_file: 200,000 · fail_on_unreadable: false
Output:   text (concatenated, each file prefixed "--- <name> ---"), files (per-file
status),
          image_files, total_files, text_file_count, image_count
```

Interview sentence: *"Right after an upload-type Start field, this turns an uploaded document reference into usable text — and keeps images separate for a vision node."*

Subprocess

Search: *(type)* `SubprocessAgent` · **Live type:** `SubprocessAgent` · **Kind:** deterministic

Why: Runs another **saved workflow** as a genuinely independent child run — not an inline sub-graph. The parent pauses (the same `interrupt()` mechanism Human Review uses) until the child finishes.

```
Required
--------
workflow: str          — the child workflow's filename, without .yaml

Optional
--------
inputs: {}             — explicit input mapping (falls back to a same-named parent
                         input, then a same-named parent node's whole output, then
None)
result_from: workflow_output (default) | node | all_outputs
result_node: str        — required if result_from: node
timeout_seconds: 1800.0
```

Output: `status` (`completed`), `result`, `child_run_id`, `child_workflow`.

Interview sentence: *"The child gets its own preflight, its own Cockpit visibility, its own audit trail — it's a real separate run, not a shortcut."*

7. Decision / Router / Multi-Route / Join — the worked examples

Two separate worked examples, because — despite how natural it sounds — **Multi-Route and Join are not meant to feed each other.** Getting this pairing wrong is a real, named preflight error in this platform, so it's worth being precise about which pattern is actually safe.

7.1 Multi-Route — several departments, each on its own path

Scenario: a message genuinely raises two separate asks — *"My pump PX-400 keeps overheating, and separately, I think I was billed twice for the last service call."* One department can't answer both; forcing it into one loses whichever ask isn't that department's.

```
Start
  ↓
Transform (ai)  "Understand Message"
  ↓
Router (mode: conditions, selection: multi)  "Which departments does this touch?"
  ├─ ENGINEERING ──→ Transform (ai)  "Draft Engineering Note" ──→ End (Engineering)
  └─ BILLING     ──→ Transform (ai)  "Draft Billing Note"     ──→ End (Billing)
```

Router config:

```yaml
mode: conditions
selection: multi
cases:
  - route: ENGINEERING
    when:
      operator: and
      conditions:
        - field: understand_message.parsed.mentions_technical_issue
          operator: is_true
  - route: BILLING
    when:
      operator: and
      conditions:
        - field: understand_message.parsed.mentions_billing_question
          operator: is_true
fallback: SUPPORT
```

Edges (identical shape to single-select — the fan-out to multiple branches is driven entirely by `selection: multi` on the node, not by anything different at the edge level):

```yaml
- from: department_router
  condition: route
  branches:
    ENGINEERING: engineering_note
    BILLING: billing_note
```

Both `engineering_note` and `billing_note` run in the same request whenever both conditions are true, each producing its **own** End result — the run's combined output then simply contains both. **Each branch ends on its own.** This is the platform's own guidance, straight from the Router editor's own UI copy: collect multi-route results as separate outputs, don't try to reconverge them.

What to say: *"A new department is still a configuration change, and now more than one can be true at once — but each one finishes its own path. Nothing downstream has to wait on a branch that might not have run."*

7.2 Join — combining branches that *always* both run

Join needs branches that are **unconditionally** parallel — not a data-dependent selection. The right source is a **plain fan-out edge** (`to: [a, b]`), which always fires every listed target, every single run — never a Multi-Route router's conditional dispatch.

Scenario: every case, regardless of content, should get both an urgency assessment and a sentiment assessment before it's filed.

```
Start → Transform (ai)  "Understand Message"
  ↓  (one plain edge, unconditional fan-out — both always run)
  ├─→ Transform (ai)  "Assess Urgency"    ──┐
  └─→ Transform (ai)  "Assess Sentiment"  ──┼─→ Join ──→ End
                                            ┘
```

Edges (plain fan-out — the `to:` list, not a router):

```yaml
- from: understand_message
  to: [assess_urgency, assess_sentiment]
- from: assess_urgency
  to: combine_assessment
- from: assess_sentiment
  to: combine_assessment
```

Join config:

```yaml
type: TextAssemblerAgent   # palette: "Join"
config:
  parts:
    - "{{outputs.assess_urgency.parsed.note}}"
    - "{{outputs.assess_sentiment.parsed.note}}"
  separator: "\n\n---\n\n"
```

This is safe because **both branches run on every single request, unconditionally** — the join always has exactly two real predecessors to wait for.

7.3 The rule, stated plainly

```
SAFE:    plain fan-out (to: [a, b])  →  both always run  →  Join waits for both,
always succeeds
UNSAFE:  Multi-Route branches        →  may not both run  →  Join could wait forever
```

Never feed two branches of one Multi-Route router into a shared Join. This is exactly what the platform's `MULTIROUTE_ANDJOIN_MAY_NOT_FIRE` preflight check exists to catch, and it's a hard error, not a warning — because a branch the router didn't select simply never runs, and a hard AND-join has no way to know to stop waiting for it. Compare also to the main demo's single-select `department_router` in §4: exactly **one** of its branches ever runs per request, so feeding two of *its* branches into a shared node hits the same family of error (`FANIN_UNREACHABLE_ANDJOIN` / `ROUTER_JOIN_UNREACHABLE`).

What to say: *"Join is built for branches that are unconditionally parallel. The moment a router — single or multi — decides which branches run, each of those branches needs its own way out, not a shared join waiting on a sibling that might never arrive."*

8. MCP Tool vs. MCP Agent

Both call tools on a connected MCP server. The difference is **who decides which tool to call.**

```
MCP Tool                                  MCP Agent
————————                                  —————————
Author picks the exact tool               Model picks the tool(s), each turn,
  at authoring time.                        in a loop, up to max_iterations.
One call. Policy-gated:                   No policy gate. No read/write
  writes need a review upstream             distinction. Every tool call the
  or an explicit unattended-write           model chooses is executed for real.
  flag.
```

MCP Tool — see the full card in §5. Use this for any specific, author-controlled, possibly-write-capable action (the CRM lookup in the main demo).

MCP Agent — search *(type)* `MCPAgent`.

```
Required
————————
objective: str        (templated — what you want the loop to figure out)

Optional
————————
model: "claude-sonnet-4-5" (default) · system_prompt · max_iterations: 5
temperature: 0.2 · allowed_tools: list[str] | None   (None = unrestricted — flagged
by preflight if unreviewed)
```

Output: `answer`, `tool_calls` (`{iteration, tool, arguments, result_preview}` per call actually made), `iterations_used`, `completed` (false if it hit `max_iterations` with no final answer).

Quick test:

```
Objective: "Look up the customer's open tickets and summarize them."
```

Expect `tool_calls` to show which tools the model actually chose, in what order — inspect this before trusting the `answer`.

Interview sentence: *"MCP Agent is for open-ended, read-only investigation where the right tool sequence genuinely isn't known in advance. The moment a write-capable tool is in scope, that decision needs to be authored and reviewed — which is exactly what MCP Tool is for instead."*

9. RAG / Knowledge Retrieval

```
Question
   ↓
Retrieve Knowledge      (KnowledgeRetrieval — retrieval only)
   ↓                        or
Grounded Context        RAGAgent — retrieval + generation, in one node
   ↓
[N]-cited AI Answer
```

KnowledgeRetrieval — search *(type)* `KnowledgeRetrieval`.

```
Required: collection_id, retrieval_profile_id, query
Optional: runtime_filters: {}
Output:   retrieved_chunks, citations (each evidence_status:
"retrieved_not_verified"),
          context (assembled text), candidate_count, context_count
```

Use this when you need retrieved chunks/citations as **data** for a downstream step (a custom prompt, a Router, a verification node) — without also generating an answer in the same step.

RAGAgent — search *(type)* `RAGAgent`.

```
Required: query
Optional: rag_agent_id (a saved retrieval+generation profile — recommended for new
          workflows) — or, if left unset, the legacy retrieval knobs directly
          (filters, top_k_candidates, top_n_final, alpha, rerank, compress)
Output:   answer, citations ([N]-labelled, each with chunk_id/source_doc/snippet),
          retrievals, rewritten_query
```

> A config validator rejects setting `rag_agent_id` **and** any legacy retrieval knob at the same time — change the saved Retrieval Profile itself instead of overriding it inline.

Quick test:

```
Query: "What is our standard refund window?"
```

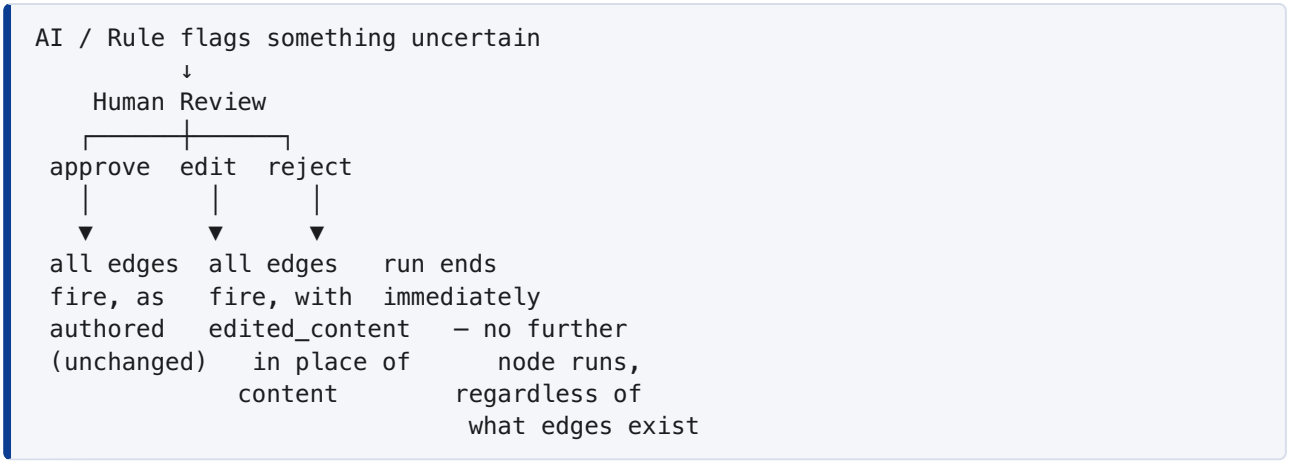
Expect `answer` to contain `[1]`-style markers and `citations` to list a real `chunk_id` / `source_doc` for each one used.

Low-confidence/no-result behavior: if retrieval returns zero chunks, only `answer` / `citations` / `retrievals` / `rewritten_query` populate — route on `citations` being empty (or `context_count == 0` for KnowledgeRetrieval) rather than assuming a populated answer.

Interview sentence: *"Every citation coming out of this platform is explicitly status-tagged 'retrieved, not verified' — this node retrieves and, with RAGAgent, drafts a cited answer, but nothing here is asserted as verified evidence on its own."*

10. Human Review — in practice

Full config card is in §5; here's the outcome flow spelled out end-to-end, since it's the one node whose behavior after each decision is easy to get wrong.



What to configure, and why each field is there: - `review_panels` over `context_fields` — a reviewer sees labelled business facts ("Customer found?", "Extraction confidence"), not a raw JSON dump of upstream state. - `review_purpose` — the one line that answers "why am I being asked this" before the reviewer even looks at the buttons. - `allowed_actions` — trim this to just `[approve, reject]` when there's genuinely nothing to *edit* (as in the main demo's escalation gate) — offering an edit action with nothing sensible to edit just adds a confusing option. - `editable_content_field` — only set this when there's real drafted content (an email body, a proposal section) a person should be able to rewrite in place.

Interview sentence: *"There's no fourth outcome and no way around it — reject always ends the run. That's deliberate: it stops a rejected case from silently continuing down a path authored for the approved one."*

11. Integrations, in practice

Three different “external” nodes, three different shapes of risk — the distinction interviewers usually probe on:

Node	Who decides the action	Policy gate	Typical use
MCP Tool	Author (one named tool)	Yes — write needs review or explicit flag	Known business-system operation
Email	Author (one operation selector)	Preflight flags `send` / `reply` with no review gate	Any mailbox interaction
Integration	Author (one operation selector)	Provider-level connection scoping	Cloud file access (Drive/ OneDrive)
External Action	Author (explicit `safety_class`)	Same review-gate expectation as a write	Any REST/webhook with no MCP connection
MCP Agent	**The model**, each turn	**None**	Open-ended read-only investigation only

Credential handling — the one non-negotiable pattern: every one of these nodes references a named **connection id** (`connection: default`, `server_id: dynamics365_finance_scm`) — never a URL, token, or API key in the workflow itself. That’s what makes a workflow exportable/shareable without leaking access. In this guide, and in your own workflows:

```
Credential: configured through the platform's connection/environment settings —
never hardcoded in the workflow.
```

Interview sentence: *“Every external system this platform touches is reached through a named connection the workflow references, never a secret the workflow contains — so the workflow file itself is safe to export, version, or hand to someone else.”*

12. Subprocesses, in practice

Full config card is in §6. The pattern worth showing live:

```
Parent Workflow                          Reusable Subprocess
─────────────────                        ──────────────────────
...
  ↓
Subprocess                      ─────→   Runs as its OWN independent
  config:                                 top-level run — own preflight,
    workflow: customer_lookup             own Cockpit visibility, own
    inputs:                               audit trail.
      customer_name: "{{...}}"
  ↓ (parent pauses here,
     via the same interrupt()
     mechanism Human Review uses)
...
Parent Continues            ←─────    result, child_run_id, child_workflow
```

Real, minimal example (`workflows/test_fixtures/generation_coverage/subprocess_agent.yaml`):

```yaml
- id: run_lookup_subprocess
  type: SubprocessAgent
  config:
    workflow: database_lookup_smoke_test
    inputs: {}
    result_from: workflow_output
- id: summarize
  type: Echo
  config:
    template: "Subprocess finished: {{outputs.run_lookup_subprocess.status}}"
```

Interview sentence: *"This isn't an inline sub-graph — it's a genuinely separate run of a saved workflow. If that child workflow changes, every parent that calls it picks up the change automatically, without being re-authored."*

13. Document & Output Nodes

Two extractors (turn an uploaded file into data) and five renderers (turn data into a deliverable file) — all Document Rendering & Export, all search by their raw class name.

Node	Direction	Required config	Produces
`ExcelTableExtractor`	in	`minio_key`	`tables` (per-sheet rows/cells), `sheet_count`, `total_rows`
`PDFTextExtractor`	in	`minio_key`	`pages` (per-page text), `page_count` — **no OCR**, a scanned PDF yields little/no text
`PDFProposalRenderer`	out	`sections`, `template` (corporate/professional/warm), `proposal_title`, `client_name`	`minio_key` (`.pdf`), `byte_size`
`PowerPointProposalSlides`	out	`sections`, `proposal_title`, `client_name`	`minio_key` (`.pptx`), `slide_count`
`DOCXProposalRenderer`	out	same as PDF renderer	`minio_key` (`.docx`) — understands a specific Markdown subset (headings, lists, pipe tables with a `---` separator, bold/italic/code)
`HorizonDOCXProposalRenderer`	out	`content`, `content_format` (markdown/html), plus metadata/citation fields	`.docx` with native TOC, page-number fields, page-limit estimate, optional evidence annex
`HorizonHTMLProposalRenderer`	out	same shape minus DOCX-specific fields	`.pdf` + its HTML source, with a **real measured** page count (via `pdfplumber`), not an estimate

Quick test (generic renderer):

```
sections: { "Executive Summary": "This proposal...", "Methodology": "..." }
template: corporate
proposal_title: "Q3 Expansion Proposal"
client_name: "Acme Corp"
```

Expect a `minio_key` back, not inline bytes — always feed it through the platform's file-download path, never assume the content is embedded in the run result.

Interview sentence: *“Prefer the Horizon-specific renderers over the generic ones the moment you need citation numbering or a hard page limit — they’re drop-in on the same* `sections` / `title` *shape, with the domain-specific checks layered on top.”*

14. Remaining Node Mini-Demos — the proposal-drafting pipelines

These 28 nodes are the platform's *other* domain: **EU Horizon-proposal drafting**. They're not "drag anywhere" primitives like Core Building Blocks — each family runs in a fixed, meaningful order, so each gets a pipeline diagram instead of a build sequence. Full field lists are in the Master Inventory (§2) and the source audit; these are the shapes and one worked snippet per family.

14.1 Research & Discovery — finding candidates

```
Call + selected concept
        ↓
ScientificResearchPlannerAgent    "Plan the research" — turns context into bounded
briefs
        ↓
   ├─ BoundedDeepResearchAgent           (multi-brief, budgeted)
   ├─ WebSearchAgent                     (one live query)
   └─ ScholarlyCandidateDiscoveryAgent   (academic databases: arXiv, OpenAlex, ...)
        ↓  (all three feed into)
ResearchSourceAcquirer            "Acquire full text" — fetches candidates into
        ↓                          immutable, hash-stamped documents
   (feeds §14.2)
```

Everything above the acquisition step produces **candidates only** — never treat a discovery-stage output as usable evidence.

Worked snippet (`ScientificResearchPlannerAgent` → `BoundedDeepResearchAgent`):

```yaml
- id: plan_research
  type: ScientificResearchPlannerAgent
  config:
    call_context: "{{inputs.call_text}}"
    concept_context: "{{outputs.freeze_concept.selected_concept}}"
    max_briefs: 6
- id: run_research
  type: BoundedDeepResearchAgent
  config:
    research_briefs: "{{outputs.plan_research.research_briefs}}"
    max_jobs: 6
    max_total_tool_calls: 60
```

Interview sentence: *"Discovery is deliberately bounded and budgeted — job count, tool calls, cost per call, duration — because this is the most expensive family of nodes in the platform per call."*

14.2 Evidence & Retrieval — verifying and serving evidence

```
Acquired documents (from §14.1)
        ↓
CitationRegistryBuilder          "Number the sources" — one stable [N] per source
        ↓
MinIOEvidenceIngestion           "Index for retrieval" — stamps chunks with [N]
        ↓
   ├─ ProposalEvidenceFactoryAgent    (full fail-closed pipeline)
   ├─ ClaimEvidenceVerifier           (passage-level verdict)
   └─ PaperQAEvidenceSynthesizerAgent (per-document coverage)
        ↓  (all three feed into)
ProposalTruthGraphAgent          "Freeze the truth graph" — only unlocks
        ↓                         drafting after a human "approve"
   (feeds drafting — RAGAgent/KnowledgeRetrieval, §9)
```

Three other nodes feed evidence into the same funnel from different sources, not through acquisition: `InternalProjectEvidenceRetrieverAgent` (partner/internal facts — requires human approval per record, never treated as external evidence), `PriorProjectRetrieverAgent` (CORDIS/LIFE/EIP-AGRI prior-art search), `StructuredDatasetRetrieverAgent` (bounded Eurostat/structured-data retrieval — every record is `human_review_required: true` by construction).

Worked snippet (the freeze gate):

```yaml
- id: freeze_truth_graph
  type: ProposalTruthGraphAgent
  config:
    verified_claims: "{{outputs.verify_evidence.result.verified_claims}}"
    evidence_approval_decision: "{{outputs.evidence_review.decision}}"   # must be
        exactly "approve"
```

Interview sentence: *“Nothing downstream can draft from evidence until a human has explicitly approved this exact snapshot — that’s the one point where a person’s sign-off unlocks drafting, and it’s enforced in code, not by convention.”*

14.3 Proposal Engineering — the drafting/gating pipeline

```
Concept note
        ↓
GraphNormalizer                          "Extract structure" — the keystone: 14 typed
        ↓                                 collections (Objectives, WorkPackages,
Claims, ...)
        ↓
ConceptAlternativesAgent                 "Generate 3 postures" — conservative/
balanced/ambitious,
        ↓                                 independently scored + adversarially judged
   Human: select and freeze
        ↓
ConceptFreezeAgent                       "Freeze the choice" — fails closed on a bad
id, no LLM call
        ↓
MethodologyEngineeringAgent              "Method Cards" — one per objective
        ↓
   (drafting happens via Transform/RAGAgent — not a dedicated node type)
        ↓
   ├─ ConsistencyChecker         (9 hard rules)
   ├─ CallCoverageMatrixAgent    (per-requirement matrix)
   └─ HorizonEvaluationAgent     (cross-provider scoring)
        ↓  (all three feed into)
ProposalSubmissionGate                   "Final release gate" — READY / BLOCKED,
                                          never calls an LLM itself
```

Worked snippet (the submission gate, wired to all three upstream signals):

```yaml
- id: submission_gate
  type: ProposalSubmissionGate
  config:
    proposal_text: "{{outputs.assemble_proposal.text}}"
    consistency_gate: "{{outputs.consistency_check.gate}}"
    evaluation_threshold_passed:
        "{{outputs.horizon_evaluation.result.threshold_passed}}"
    evaluation_total_score: "{{outputs.horizon_evaluation.result.total_score}}"
```

Interview sentence: *"A high evaluator score can never hide missing evidence or a failed consistency check — the gate combines every hard, auditable signal itself, and it's the one node here that deliberately never calls a model."*

14.4 Multimodal

```
Uploaded image                      Text prompt                  Drafted content with
        ↓                                 ↓                       [[IMAGE
PROMPT: ...]] markers
KimiVisionAgent              OpenAIImageGenerationAgent                  ↓
"Analyze the image"          "Generate one image"              DynamicFigureAgent
        ↓                                 ↓                       "Illustrate the
draft" —
   analysis (text)              minio_key (stored reference)     scans every marker,
generates
                                                                  + inlines each as
base64
```

`FigureEmbedder` is the older, explicit-marker pattern (a fixed list of `{marker, image}` pairs) that `DynamicFigureAgent` supersedes for new work — still registered and functional, prefer the newer node.

Worked snippet:

```yaml
- id: analyze_photo
  type: KimiVisionAgent
  config:
    image: "{{inputs.uploaded_photo}}"
    prompt: "Describe the visible damage to this pump housing."
```

Interview sentence: *"Image bytes never touch workflow state directly — every one of these nodes hands back a storage reference or, for DynamicFigureAgent, an inline base64 URI built specifically for the DOCX renderer to embed."*

14.5 The remaining Integrations-family nodes

SQL Query Agent — search *(type)* `SQLQueryAgent`. A read-only escape hatch when a classified MCP tool doesn't cover a lookup — **no write mode exists at all**.

```yaml
- id: lookup_open_orders
  type: SQLQueryAgent
  config:
    server_id: business_records
    sql: "SELECT order_id, status FROM orders WHERE customer_id = %(customer_id)s"
    params:
      customer_id: "{{outputs.find_customer.first.account_id}}"
    max_rows: 50
```

Real safety (read-only credentials, row/timeout limits, write-verb rejection) lives server-side, not in this node's config.

Python Snippet Agent — search *(type)* `PythonSnippetAgent`. Runs a short snippet in a genuinely network-isolated sidecar process (`network_mode: none`, non-root, capabilities dropped) — reached over a Unix socket, never `exec()`'d in-process.

```yaml
- id: compute_lead_time
  type: PythonSnippetAgent
  config:
    code: "output = {'weeks': max(2, inputs['quantity'] // 50)}"
    input_fields:
      quantity: "{{outputs.understand_message.parsed.quantity}}"
    output_fields:
      - name: weeks
        type: integer
```

Interview sentence: *"This runs in an isolated sidecar with no network access — a naive in-process attempt at this exact capability once leaked an API key and froze the event loop, which is exactly why it doesn't work that way anymore."*

Scientific Skill Agent — search *(type)* `ScientificSkillAgent`. One LLM completion augmented with curated, versioned guidance from the platform's Scientific Agent Skill catalog — the model is explicitly told the skill text is guidance only, never permission to bypass review/evidence controls.

```yaml
- id: draft_methodology_note
  type: ScientificSkillAgent
  config:
    objective: "Explain the statistical power analysis behind this trial design."
    auto_select: true
    max_skills: 2
```

Interview sentence: *“This is one-shot and never retrieves anything itself — pair it with a research/retrieval node upstream whenever it needs facts that aren’t already in context.”*

15. Testing Cheat Sheet — the main demo (§4)

Run each of these through the Builder's **Simulate** tab (whole workflow, one example, lights up the executed path — no Run History record) or the **Run in Cockpit** button (a real run). Use the Inspector's **Test** tab to check one node in isolation before wiring the next.

Happy path

```
message: "Our PX-400 keeps overheating after 20 minutes. – Anita Rao, Acme
Fabrication"
customer_name: "Acme Fabrication"     (a name that exists in the connected system)
```

Expect: `intent: TECHNICAL_ISSUE`, `confidence` ≥ 0.6, `find_customer.found: true`, `department: ENGINEERING`, routed straight to the Engineering `End` — no escalation.

Missing information

```
message: ""    (leave the required Start field empty)
```

Expect: the run never starts — `REQUIRED_INPUT_MISSING`. This is the one failure mode caught before any node executes at all, not mid-run.

Low AI confidence

```
message: "hi there question"
```

A message this thin should genuinely produce a low `confidence`. Expect `department` overridden to `NEEDS_REVIEW` by the second escalation rule, landing on Human Review regardless of whatever `intent` the model guessed.

Multiple routes (§7.1)

```
message: "My PX-400 keeps overheating, and separately I was billed twice last
month."
```

On the Multi-Route variant: expect **both** `engineering_note` and `billing_note` to have run in the same simulation — check the Simulator's highlighted path shows both branches lit, not just one.

Integration failure

Test `find_customer` in isolation with a name that doesn't exist in the connected system:

```
customer_name: "Not A Real Company Ltd"
```

Expect `status: ok` (not an error) with `found: false, count: 0` — `fail_on_error: false` is what makes this a routable fact instead of a hard stop. As a contrast test, temporarily flip `fail_on_error: true` and confirm the run now fails outright on the same input — this is the exact tradeoff to be able to explain live.

Human reject / edit

On the escalation branch, resume the paused run three ways and confirm each behaves as documented in §10: - **Approve** → Email drafts, `End` produces a result. - **Edit** the reviewer's content → Email drafts with the edited text, not the original. - **Reject** → the run ends immediately. No Email node runs. There is no edge to check for this — that's the point.

Join (§7.2)

Run the "always assess urgency and sentiment" variant and confirm `all_parts_present: true` on every run — since both branches are unconditional, this should never come back `false`. If it ever does, that's a real bug (a branch failed silently), not expected behavior — unlike Multi-Route, where an unselected branch never running is normal.

16. Complete Node Coverage Matrix

Live Node	UI Authorable	Demo Eligible	Main Workflow (§4)	Mini/ Worked Demo	Guide Section	Covered
StartAgent	Yes	Yes	Yes	—	§4, §5	Yes
EndAgent	Yes	Yes	Yes	—	§4, §5	Yes
TransformAgent	Yes	Yes	Yes (both modes)	§7.1, §7.2	§4, §5	Yes
DecisionAgent	Yes	Yes	Yes	—	§4, §5	Yes
RouterAgent	Yes	Yes	Yes (single)	§7.1 (multi)	§4, §5, §7	Yes
HumanInLoopAgent	Yes	Yes	Yes	—	§4, §5, §10	Yes
EmailAgent	Yes	Yes	Yes	—	§4, §5, §11	Yes
MCPToolAgent	Yes	Yes	Yes	—	§4, §5, §8	Yes
IntegrationAgent	Yes	Yes	No	Field card	§5, §11	Yes
ExternalActionAgent	Yes	Yes	No	Field card	§5, §11	Yes
Literal	Yes	Yes	No	Field card	§6	Yes
Echo	Yes	Yes	No	Field card + used in real snippets	§6	Yes
WorkflowFileLoader	Yes	Yes	No	Field card	§6	Yes
TextAssemblerAgent (Join)	Yes	Yes	No	§7.2 worked example	§6, §7	Yes
SubprocessAgent	Yes	Yes	No	§12 worked example	§6, §12	Yes
MCPAgent	Yes	Yes	No	§8 comparison + example	§8	Yes
RAGAgent	Yes	Yes	No	§9 worked example	§9	Yes
KnowledgeRetrieval	Yes	Yes	No	§9 worked example	§9	Yes
ExcelTableExtractor	Yes	Yes	No	§13 table	§13	Yes
PDFTextExtractor	Yes	Yes	No	§13 table	§13	Yes
PDFProposalRenderer	Yes	Yes	No	§13 table + snippet	§13	Yes
PowerPointProposalSlides	Yes	Yes	No	§13 table	§13	Yes
DOCXProposalRenderer	Yes	Yes	No	§13 table	§13	Yes

Live Node	UI Authorable	Demo Eligible	Main Workflow (§4)	Mini/ Worked Demo	Guide Section	Covered
HorizonDOCXProposalRenderer	Yes	Yes	No	§13 table	§13	Yes
HorizonHTMLProposalRenderer	Yes	Yes	No	§13 table	§13	Yes
ScientificResearchPlannerAgent	Yes	Yes	No	§14.1 pipeline + snippet	§14.1	Yes
BoundedDeepResearchAgent	Yes	Yes	No	§14.1 pipeline + snippet	§14.1	Yes
WebSearchAgent	Yes	Yes	No	§14.1 pipeline	§14.1	Yes
ScholarlyCandidateDiscoveryAgent	Yes	Yes	No	§14.1 pipeline	§14.1	Yes
ResearchSourceAcquirer	Yes	Yes	No	§14.1 pipeline	§14.1	Yes
CitationRegistryBuilder	Yes	Yes	No	§14.2 pipeline	§14.2	Yes
MinIOEvidenceIngestion	Yes	Yes	No	§14.2 pipeline	§14.2	Yes
ProposalEvidenceFactoryAgent	Yes	Yes	No	§14.2 pipeline	§14.2	Yes
ClaimEvidenceVerifier	Yes	Yes	No	§14.2 pipeline	§14.2	Yes
PaperQAEvidenceSynthesizerAgent	Yes	Yes	No	§14.2 pipeline	§14.2	Yes
ProposalTruthGraphAgent	Yes	Yes	No	§14.2 pipeline + snippet	§14.2	Yes
InternalProjectEvidenceRetrieverAgent	Yes	Yes	No	§14.2 note	§14.2	Yes
PriorProjectRetrieverAgent	Yes	Yes	No	§14.2 note	§14.2	Yes
StructuredDatasetRetrieverAgent	Yes	Yes	No	§14.2 note	§14.2	Yes
GraphNormalizer	Yes	Yes	No	§14.3 pipeline	§14.3	Yes
ConceptAlternativesAgent	Yes	Yes	No	§14.3 pipeline	§14.3	Yes
ConceptFreezeAgent	Yes	Yes	No	§14.3 pipeline	§14.3	Yes
MethodologyEngineeringAgent	Yes	Yes	No	§14.3 pipeline	§14.3	Yes
ConsistencyChecker	Yes	Yes	No	§14.3 pipeline	§14.3	Yes

Live Node	UI Authorable	Demo Eligible	Main Workflow (§4)	Mini/ Worked Demo	Guide Section	Covered
CallCoverageMatrixAgent	Yes	Yes	No	§14.3 pipeline	§14.3	Yes
HorizonEvaluationAgent	Yes	Yes	No	§14.3 pipeline	§14.3	Yes
ProposalSubmissionGate	Yes	Yes	No	§14.3 pipeline + snippet	§14.3	Yes
KimiVisionAgent	Yes	Yes	No	§14.4 pipeline + snippet	§14.4	Yes
OpenAIImageGenerationAgent	Yes	Yes	No	§14.4 pipeline	§14.4	Yes
DynamicFigureAgent	Yes	Yes	No	§14.4 pipeline	§14.4	Yes
FigureEmbedder	Yes	Yes	No	§14.4 note (superseded)	§14.4	Yes
SQLQueryAgent	Yes	Yes	No	§14.5 snippet	§14.5	Yes
PythonSnippetAgent	Yes	Yes	No	§14.5 snippet	§14.5	Yes
ScientificSkillAgent	Yes	Yes	No	§14.5 snippet	§14.5	Yes
WorkflowInputAgent	**No (hidden)**	No	No	Exclusion note	§5	Yes*
AITaskAgent	**No (hidden)**	No	No	Exclusion note	§5	Yes*
DataTransformAgent	**No (hidden)**	No	No	Exclusion note	§5	Yes*

57 / 57 registered node types accounted for. *The 3 hidden nodes are “covered” in the sense the guide names them, explains the exclusion, and points to their live replacement — never silently omitted — but they are correctly excluded from every “what to drag” instruction, per the platform’s own product intent.

Excluded-node detail (per the platform’s own hiding decision, not this guide’s judgment):

```
Node:      WorkflowInputAgent
Reason:    Superseded by StartAgent; kept registered only so already-saved
           workflows using it still open and run correctly.
Evidence:  ui/src/modes/studio/NodePalette.tsx, HIDDEN_FROM_PALETTE set;
           app/nodes/start.py module docstring names it as the successor.

Node:      AITaskAgent
Reason:    Superseded by TransformAgent's mode: ai; the class itself is
           marked deprecated in its own `about` metadata.
Evidence:  ui/src/modes/studio/NodePalette.tsx, HIDDEN_FROM_PALETTE set;
           app/nodes/ai_task.py `about` dict states the deprecation directly.

Node:      DataTransformAgent
Reason:    Superseded by TransformAgent's mode: deterministic — identical
           14-operation grammar, now under one node with the AI mode.
Evidence:  ui/src/modes/studio/NodePalette.tsx, HIDDEN_FROM_PALETTE set;
           app/nodes/data_transform.py `about` dict states the deprecation.
```

No specialized (non-Core) node type was excluded from this guide — all 28 have a section, none were judged “doesn’t fit” without a mini-demo, per §14.

17. Interview Talking Points

Short, sayable lines for the moments that come up:

- **"Why not just one big AI prompt?"** → *"Because a rule that's readable, testable, and gives the same answer every time is worth more than a model's best guess for anything the business can actually write down as a policy — Decision and Router carry that logic; AI understands the unstructured part."*
- **"How do you handle uncertainty?"** → *"Every AI step in this platform reports a confidence, and Decision/Router can gate on it directly — an uncertain case routes to Human Review instead of proceeding on a guess."*
- **"What stops this from sending something it shouldn't?"** → *"Every external write — email, MCP tool, REST call — is either behind an explicit review gate or an explicit author sign-off. It's checked against real completed node outputs in that run, not just asserted in config."*
- **"How do credentials work?"** → *"Every external node references a named connection, never a token or URL. The workflow file itself never contains a secret, so it's safe to export or version."*
- **"What if two things need to happen at once?"** → *"Multi-Route lets more than one branch fire from the same decision — each finishes on its own. Join is for branches that are unconditionally parallel from the start, not a router's conditional selection — mixing those two up is one of the platform's actual named validation errors."*
- **"How do you extend this without writing code?"** → *"Almost everything is a configuration change to Transform, Decision, Router, or MCP Tool. A new node type only gets written when there's a genuinely new integration surface — document rendering, evidence verification — not a new business rule."*
- **"How do you know a workflow is actually safe to run?"** → *"Preflight runs on every save, with no model call — it validates graph topology, every template reference, every rule's operator-vs-field-type match, and specifically checks for exactly the join/multi-route hazard above, before a single node executes."*

18. The Companion Workflow YAML

The full 14-node “Customer Request Routing” workflow from §4 is saved as real, working YAML at:

```
workflows/test_fixtures/interview_demo_customer_request_routing.yaml
```

It uses exact live node identifiers, the platform’s real edge/branch/mapping syntax throughout, a real `experience:` block on every node (required for Guided Run visibility), and the same `server_id: dynamics365_finance_scm` MCP connection the platform’s own production pump-routing workflows use. It has been run through the repository’s own preflight validator:

```
$ python scripts/preflight_workflows.py workflows/test_fixtures/
interview_demo_customer_request_routing.yaml
PASS workflows/test_fixtures/interview_demo_customer_request_routing.yaml — 14
nodes, 0 errors, 0 warnings, 0 tokens
```

It’s saved under `test_fixtures/` — the repository’s existing convention for demo/teaching workflows not intended for production use — rather than the top level, which this guide’s own audit found is reserved for the platform’s real production and flagship-demo workflows.

19. Validation Checklist

```
[x] Every palette name in this guide exists (verified directly against
    ui/src/modes/studio/NodePalette.tsx's PALETTE_LABELS and HIDDEN_FROM_PALETTE).
[x] Every documented live type exists (verified against the 57-class registry
    in app/nodes/registry.py — 1:1, zero drift).
[x] Every required configuration field exists (verified against each node's
    real Pydantic config_schema, not paraphrased from older docs).
[x] Every input/output name exists (verified against each node's real
    input_schema/output_schema).
[x] Every mapping uses supported syntax ({{outputs.<node>.<field>}}, the
    trailing "?" for optional, verified against app/runtime/templating.py).
[x] Every connection is structurally valid (verified by running preflight
    on the companion YAML, not just by inspection).
[x] Every Decision rule shape matches the real Rule/ConditionGroup schema.
[x] Every Router configuration (field/conditions/multi) matches RouterConfig.
[x] Every Multi-Route example avoids the compiler's actual dispatch mechanics
    (§7.1) — corrected once, mid-authoring, after catching a design mistake
    that would have hit a real preflight error (see §7.3).
[x] Every Join configuration matches the runtime's actual AND-join behavior
    (app/runtime/compiler.py's _wire_edges) — not guessed.
[x] The one integration referenced (dynamics365_finance_scm / find_customer)
    is a real server_id/tool pair copied from a real production workflow
    (workflows/pump_manufacturer_case_routing.yaml), not invented.
[x] No subprocess is referenced in the main demo (out of scope for Core
    Building Blocks) — the real pattern is shown in §12 instead, copied
    verbatim from workflows/test_fixtures/generation_coverage/subprocess_agent.yaml.
[x] No secrets are hardcoded anywhere in this guide or the companion YAML —
    every external node references a named connection only.
[x] No unrestricted SQL is provided — the SQLQueryAgent example (§14.5) uses
    a single parametrized SELECT, matching the node's read-only design.
[x] The companion workflow passes structural preflight: PASS, 0 errors, 0 warnings.
[x] Every one of the 57 live, registered node types appears in this guide —
    see the coverage matrix in §16.
```

This guide, the companion workflow YAML, and the fixture files it required (`workflows/test_fixtures/generation_coverage/integration_agent.yaml`, `.../web_search_agent.yaml`) were all produced by reading the live source directly — `app/nodes/*.py`, `app/runtime/*.py`, `ui/src/modes/studio/*.tsx` — and cross-checked against the repository's own preflight validator and test suite, not against prior documentation where the two disagreed.